

NLP

RNN LSTM y GRU

Docente:
Josselyn Ordóñez

Timeline



1990: Celda RNN básica (Elman)
1997: LSTM

1957: "you shall know a word by the company it keeps" (J.R. Firth)



1990s-2000s:
bag of words

2008: Collobert & Weston show value of pre-trained embeddings

2014: GloVe embeddings and seq2seq models

2018: Transformer, ELMo, ULM-FIT, BERT



19
57

19
75

19
90s

20
03

20
08

20
13

20
14

20
16

20
18

20
20s

1975: Salton's vector space model

2003: Bengio et al. train 1st neural language model, coin *word embeddings*

2013: Mikolov proposes word2vec for quick training of embeddings

2016: Fasttext library: vectors as bags of character n-grams

2020s: Large Language Models breakout: GPTs, LaMDA, BLOOM, PaLM, GLaM

2016: LSTMs se convierten en SotA para traducción automática

Redes Neuronales Recurrentes (RNNs)



Es un tipo de neurona con un estado interno (o memoria) de manera que la información del pasado influye en los resultados futuros.



Se utiliza principalmente para resolver problemas de secuencia, en donde el valor anterior está relacionado con el valor futuro.



Permite construir modelos cuyos tamaños de secuencia sean potencialmente arbitrarios.

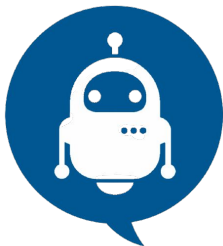


Implementa modelos de lenguaje de la forma:

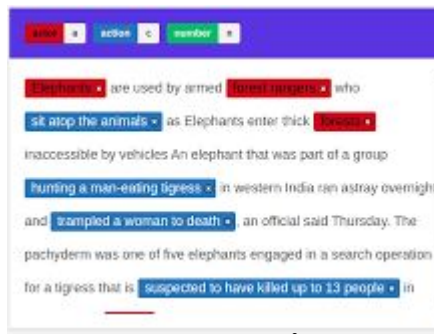
$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

*"Hoy el **día** está **hermoso** y **despejado**, se puede ver un hermoso **cielo... azul**"*

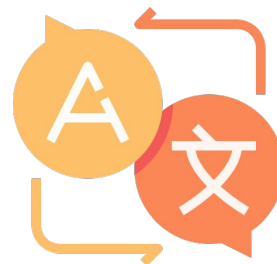
Algunos problemas de secuencia



Bots
Conversacionales



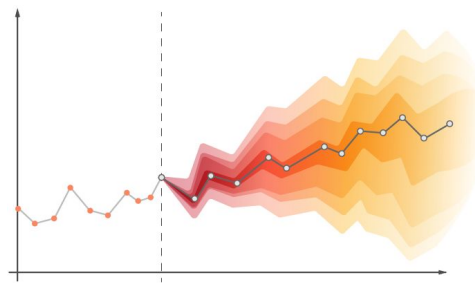
Name entity
recognition



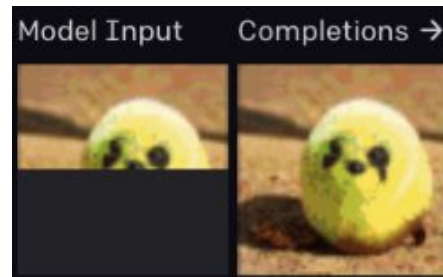
Traducción de
idiomas



Speech to text



Series
temporales



Completar una
imagen

Celda RNN básica (Elman)

[LINK](#)

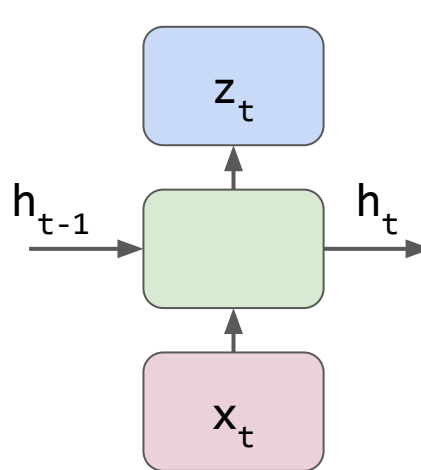
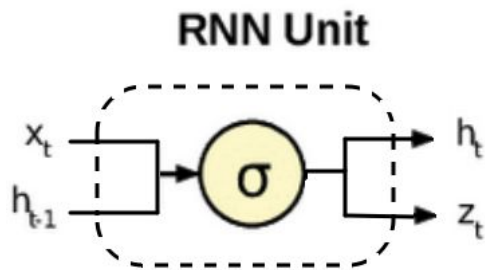
[API KERAS](#)



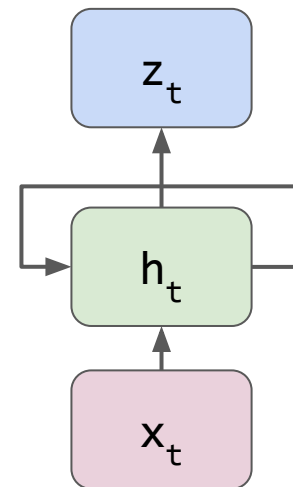
En Keras: SimpleRNN

$$h_t = \sigma(W_{hh} * h_{t-1} + W_{hx} * x + b_h)$$

$$z_t = h_t$$



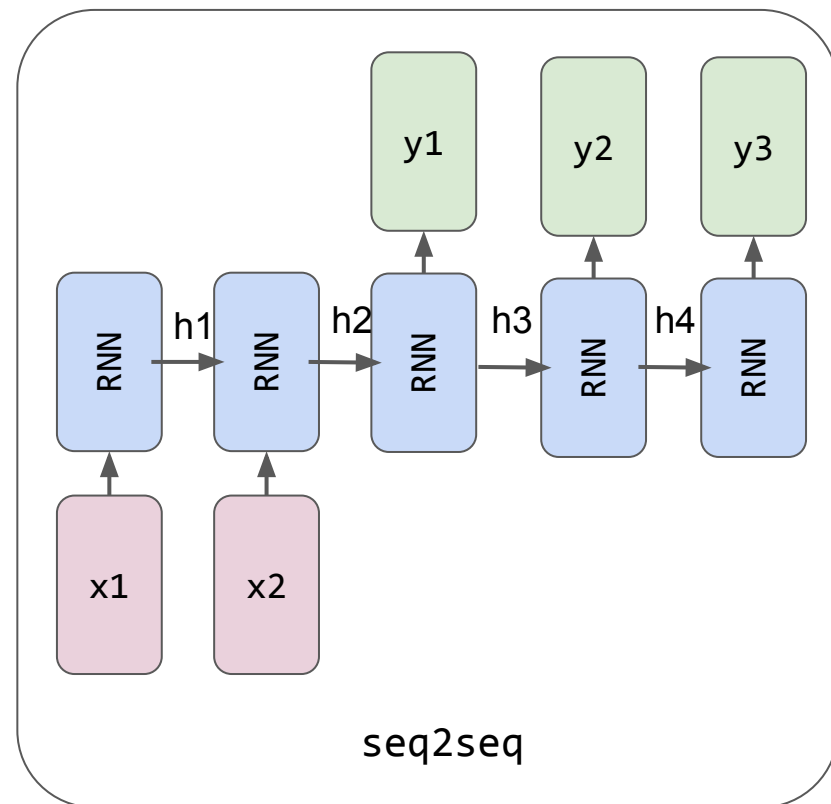
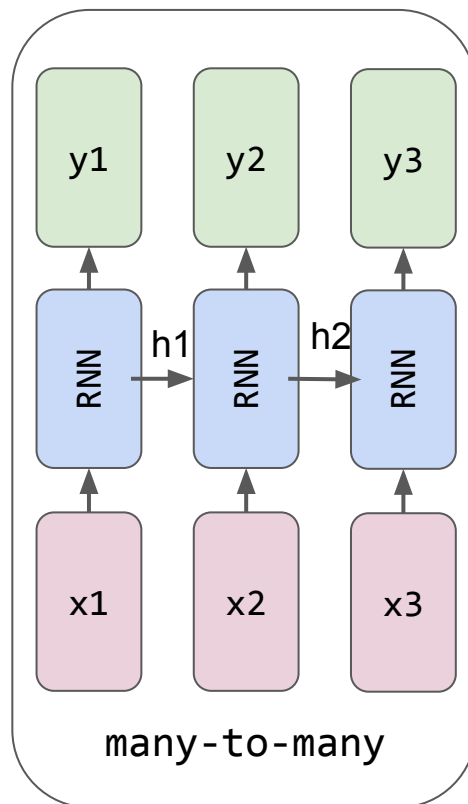
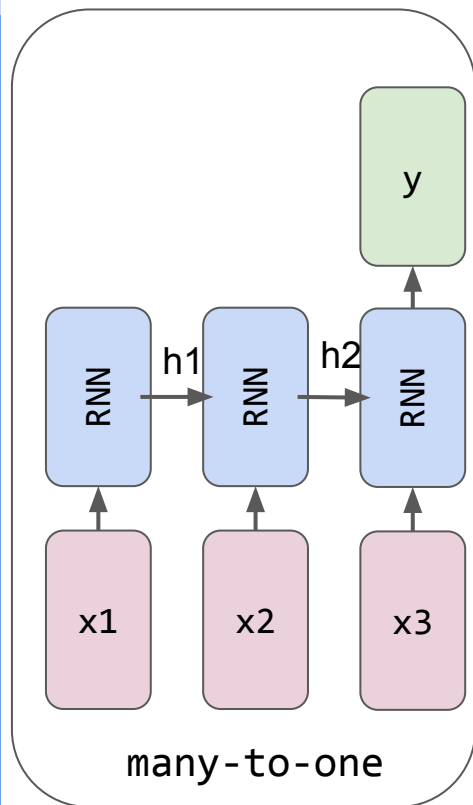
Unidad básica



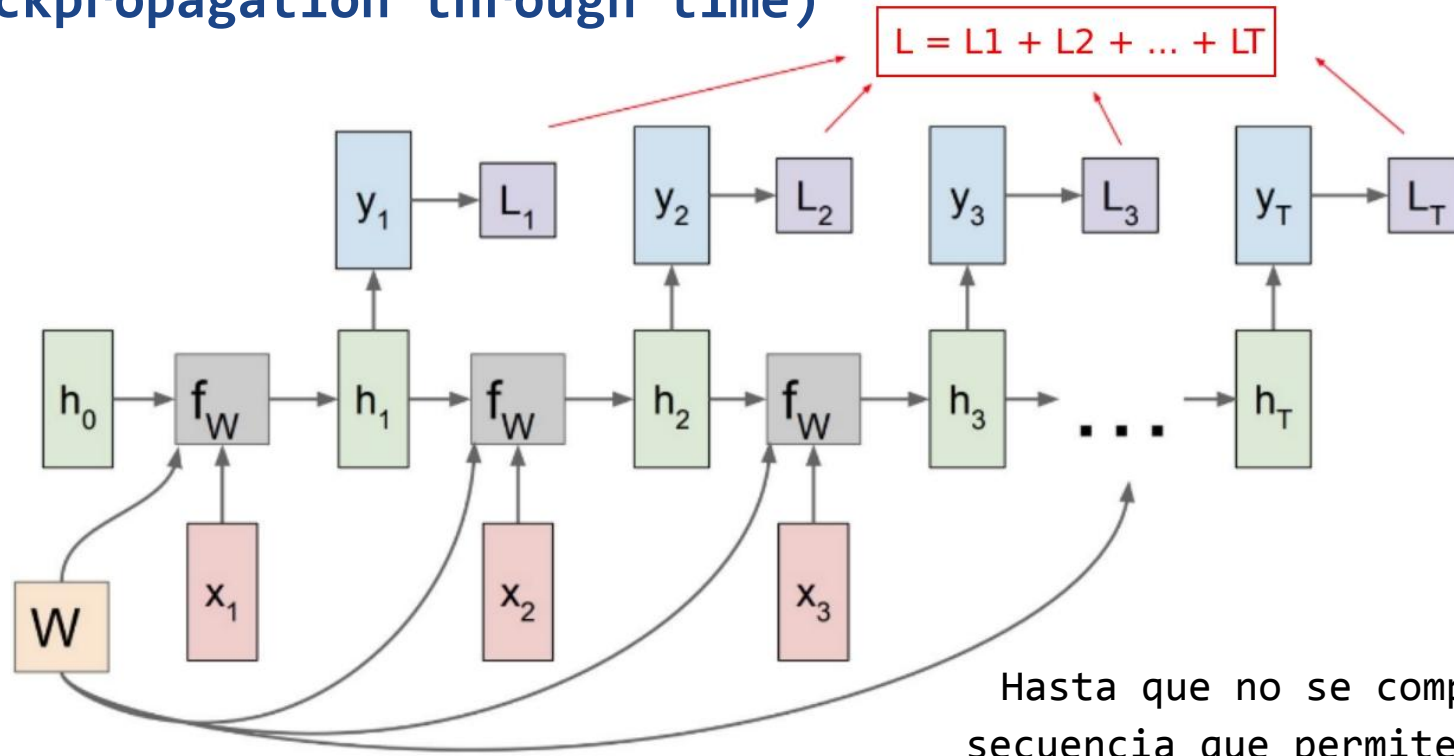
Representación
compacta

Tipos de problemas de secuencia

[LINK](#)



Grafo de cómputo de una RNN y BPTT (Backpropagation through time)

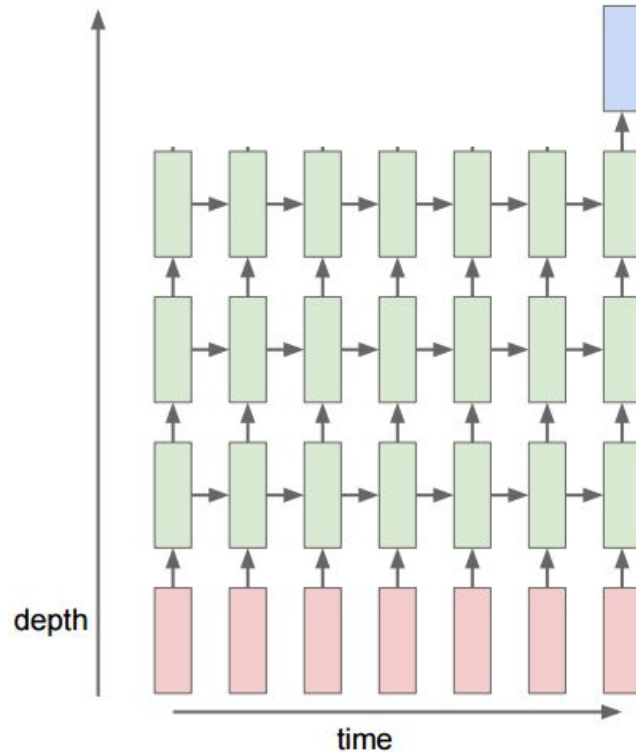


Hasta que no se complete la
secuencia que permite calcular
el loss no se actualizan los
pesos (W) de la/s celda/s

Multi-layer RNN



Tal como se vio en los ejemplos se trata de apilar layers RNN en donde la salida de una se traslada a la entrada de la siguiente



Problema de una RNN tradicional

[LINK](#)



"Una RNN tradicional solo usa información del pasado y no de las futuras palabras para predecir"

Ejemplo: Data una sentencia determinar si existe una entidad que represente al nombre de una persona utilizando (name entity recognition)

Ejemplo 1:

*"Hoy escuche que **Victoria** terminó su bot para NLP" → persona*

Ejemplo 2:

*"Hoy escuche que **Victoria** cambió de intendente" → ciudad*

La
contextualización
de la palabra es
a futuro

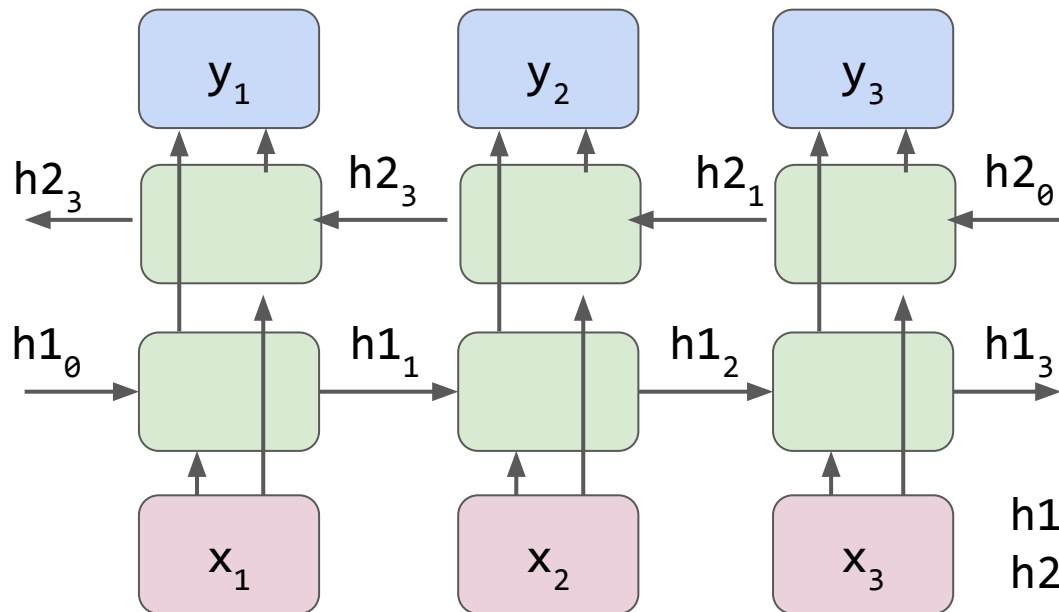
Bidirectional RNN (BRNN)

[BRNN PAPER](#)

[API KERAS](#)



“La palabra anterior y la palabra futura tienen impacto en la presente predicción”



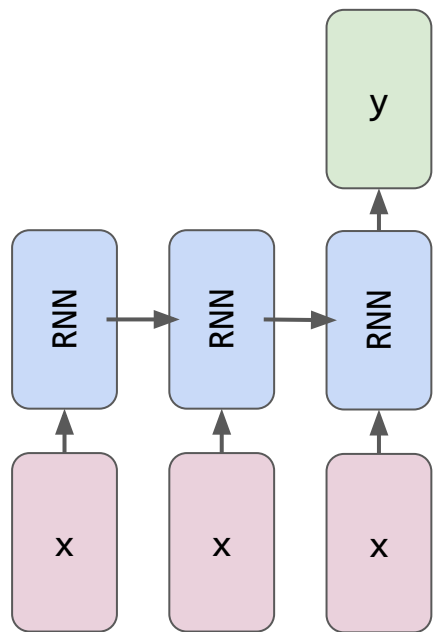
$$h1_t = \sigma(W_{hh1} * h1_{t-1} + W_{hx1} * x + b_1)$$
$$h2_t = \sigma(W_{hh2} * h2_{t+1} + W_{hx2} * x + b_2)$$

Las dos salidas pueden concatenarse, sumarse o promediarse

many-to-one



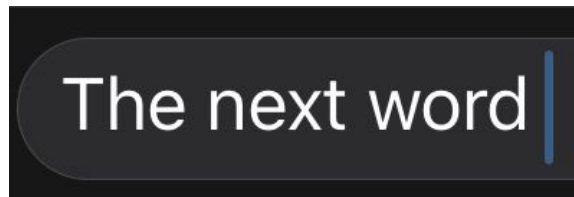
"Dada una sentencia o oración de entrada de tamaño fijo, el sistema arroja un único resultado que la representa".



many-to-one



Este tipo de estructuras se utilizan para determinar cuál es la siguiente palabra o elemento en la secuencia o para clasificación (sentiment analysis).



Predicción de próxima palabra



Análisis de sentimientos

Long short term memory (LSTM)

[LINK](#)



Se introduce este tipo de celda neuronal con mayor persistencia de memoria para lograr capturar relaciones de palabras a largo plazo.



Se crearon en 1997. Se adoptó como la layer principal para problemas de secuencia en 2014 hasta la aparición de los transformers en 2017.



Desplazaron completamente a las capas RNN simples (Elman), ya que el costo adicional de las LSTM es marginal respecto al beneficio que otorgan

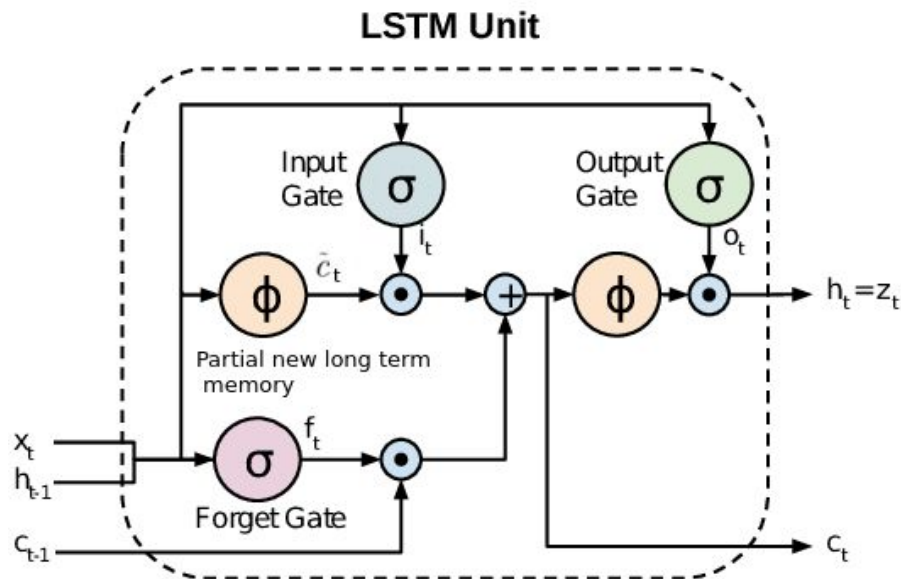


Se basan en el principio de ponderar la importancia de una palabra respecto al contexto futuro/pasado (key words).

¿Comprarías este producto?

*"**Incredible!** El producto es lo que venden, hace lo que tiene que hacer y me **ayudó mucho** a resolver los problemas que tenía. Lo **volvería a comprar** sin dudas"*

LSTM approach - Idea de la arquitectura



C: 'Memoria a largo plazo'

Input (i): Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget (f): Dada la entrada (X) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

Output (o): Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

$\cdot$: Producto elemento a elemento

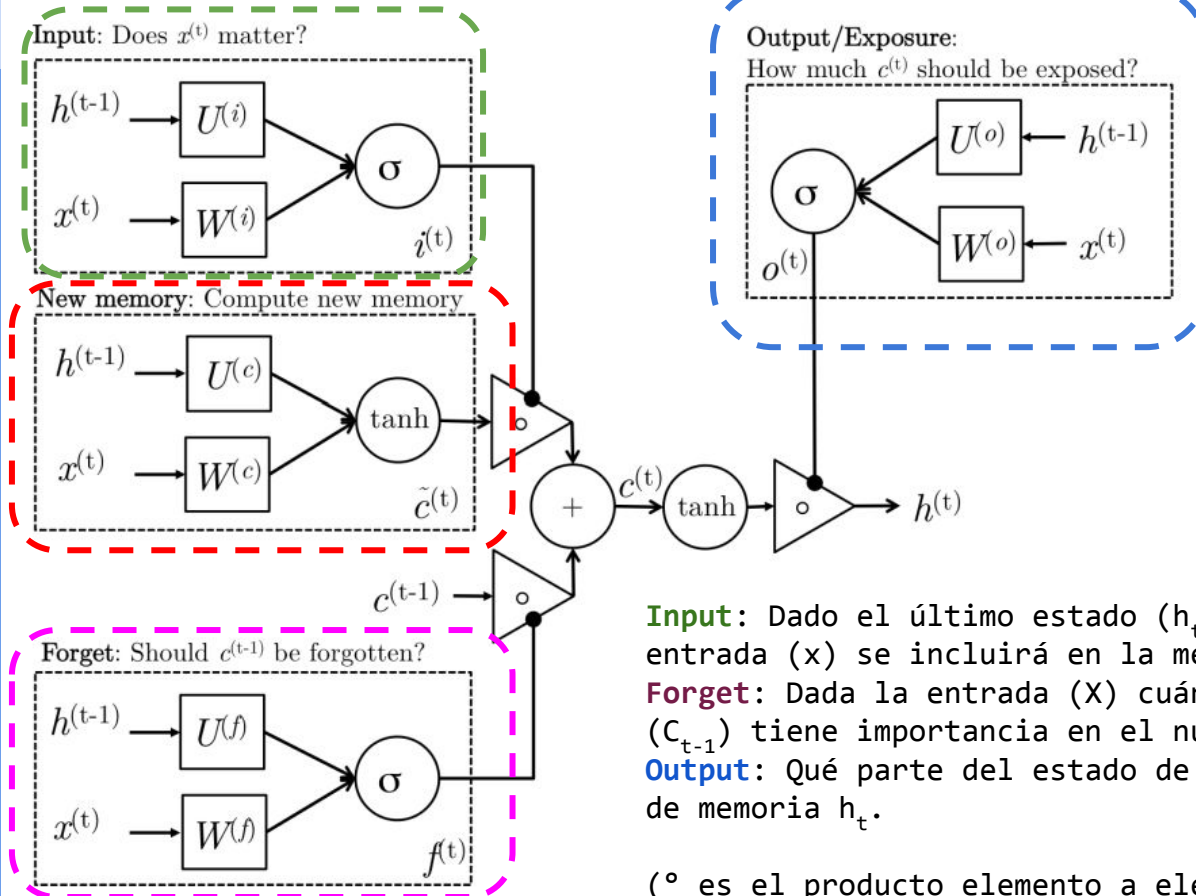
ϕ : Función Tanh

$+$: Suma vectorial

$\rightarrow$: Conexiones

LSTM approach

[LINK](#)



$$i_t = \sigma(W^{(i)} x_t + U^{(i)} h_{t-1})$$

$$f_t = \sigma(W^{(f)} x_t + U^{(f)} h_{t-1})$$

$$o_t = \sigma(W^{(o)} x_t + U^{(o)} h_{t-1})$$

$$\tilde{c}_t = \tanh(W^{(c)} x_t + U^{(c)} h_{t-1})$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

Input: Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget: Dada la entrada (x) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

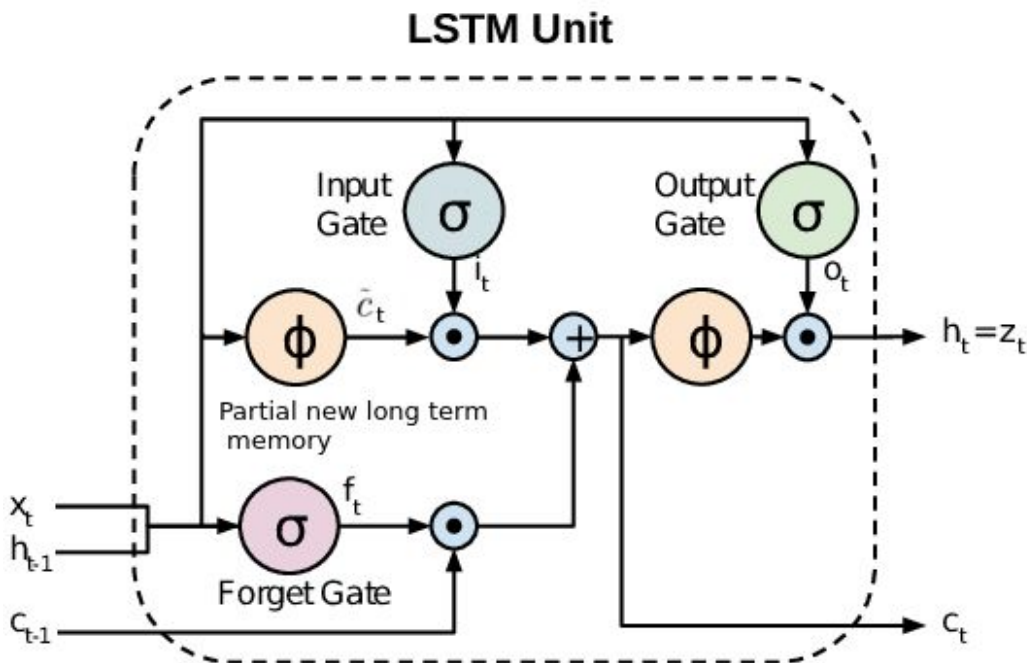
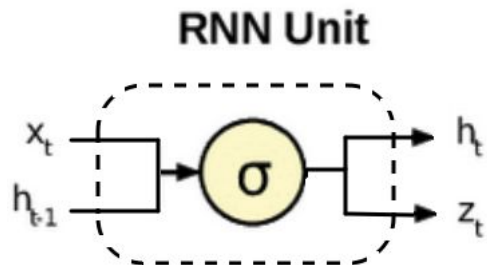
Output: Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

($\circ$ es el producto elemento a elemento)

LSTM vs RNN

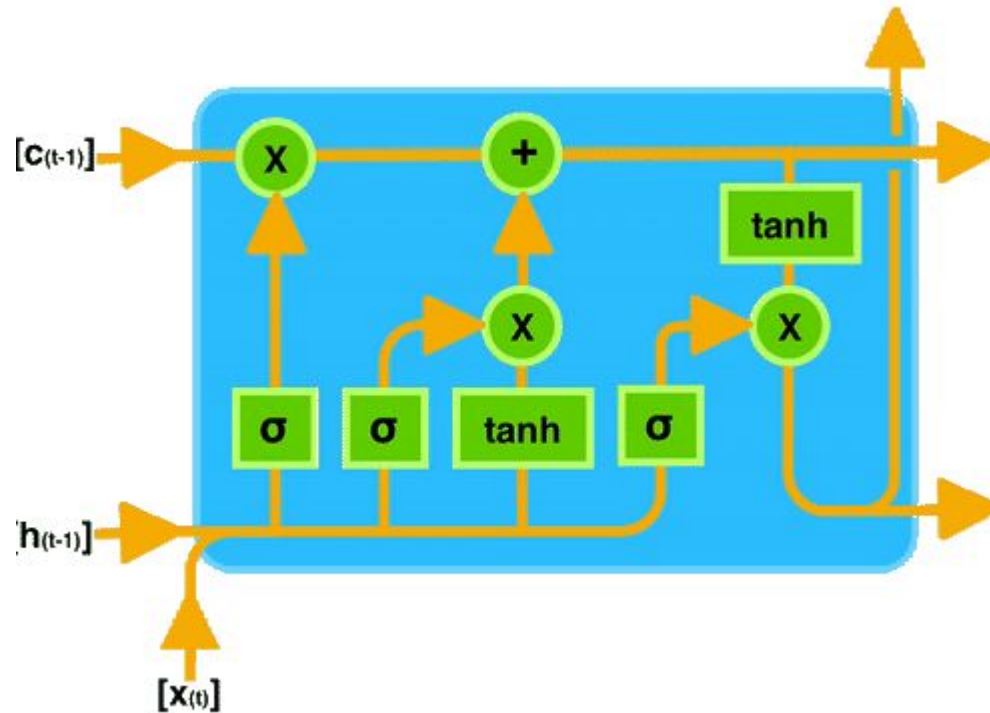


La memoria de largo plazo c_t permite propagar gradiente eficientemente a mayor “profundidad temporal”.



Long short term memory (LSTM)

[LINK](#)

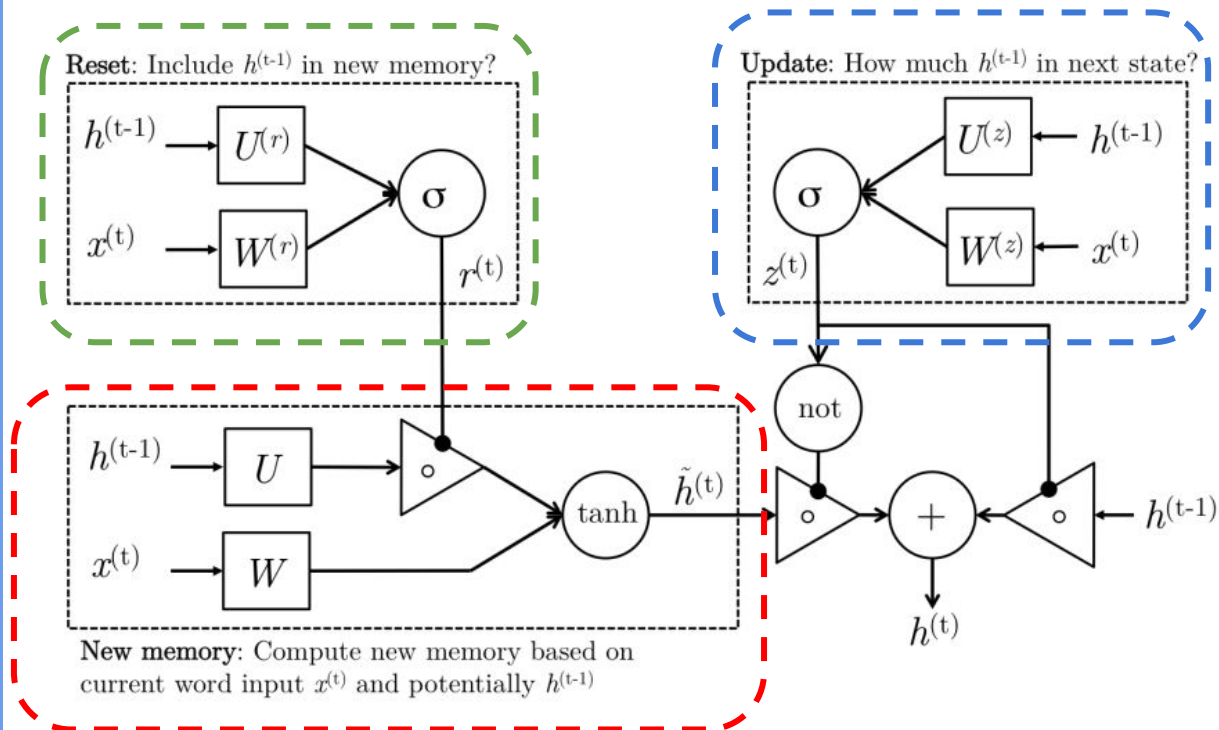


Gated Recurrent Units (GRU)

[LINK](#)



"Evolución de las RNN para superar problemas de "short-memory", versión reducida de una LSTM".



$$\begin{aligned} z_t &= \sigma(W^{(z)}x_t + U^{(z)}h_{t-1}) \\ r_t &= \sigma(W^{(r)}x_t + U^{(r)}h_{t-1}) \\ \tilde{h}_t &= \tanh(r_t \odot U h_{t-1} + W x_t) \\ h_t &= (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \end{aligned}$$

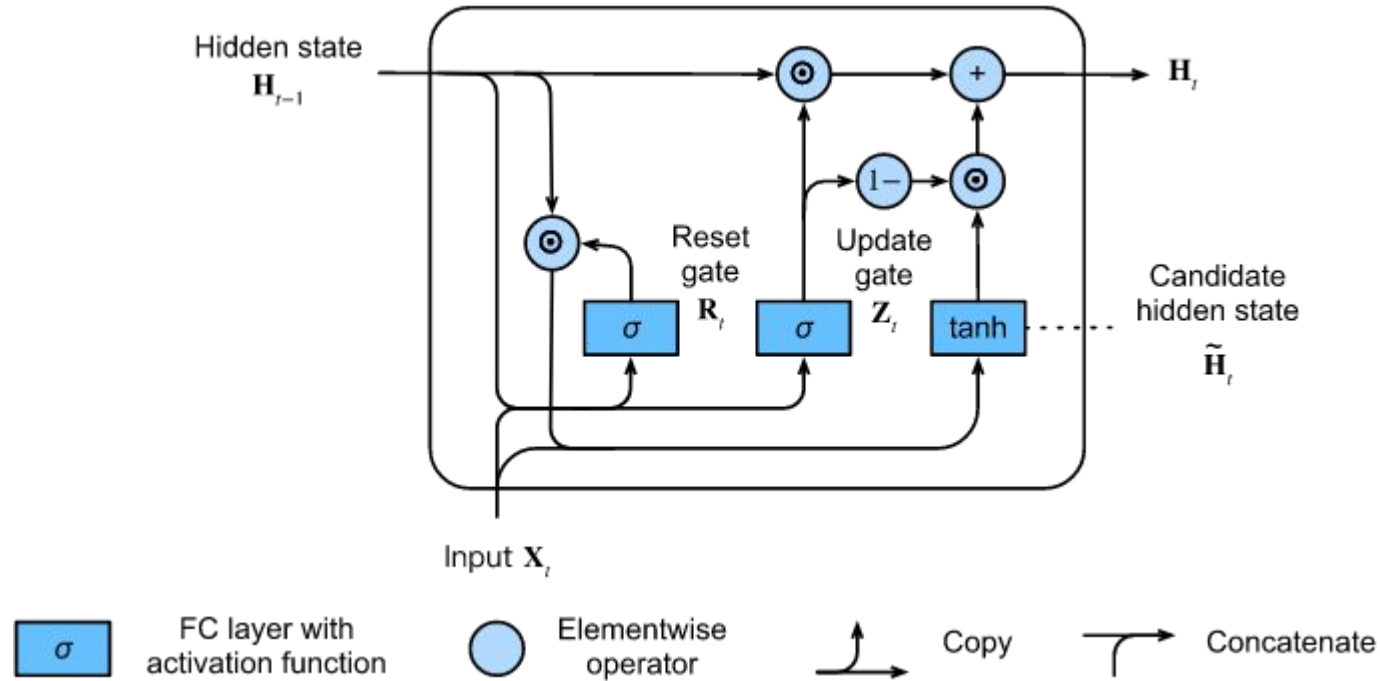
(Update gate)

(Reset gate)

(New memory)

(Hidden state)

Gated Recurrent Units (GRU)



Predicción de texto - Modelos de lenguaje



Objetivo: Entender la estructura estadística del lenguaje aprovechando su estructura secuencial.

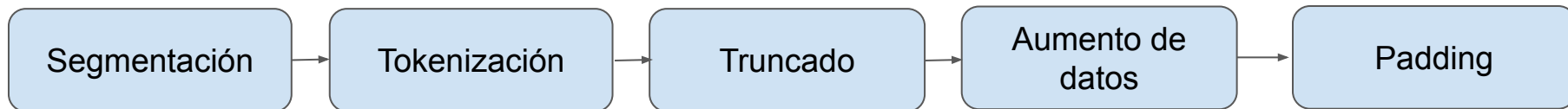
Tarea: Aprender a predecir la siguiente palabra.

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

The next word |

$$(X_1, \dots, X_n) \longrightarrow X_{n+1}$$

Pasos a seguir:



Segmentación y Tokenización



Documentos:

`'Yesterday, all my troubles seemed so far away'`

Segmentación:

`['yesterday', 'all', 'my', 'troubles', 'seemed', 'so', 'far', 'away']`

Tokenización:

→ `[ 34, 189, 200, 1345, 8, 69, 12,753]`

Ventana de contexto



La **ventana de contexto** en un modelo de lenguaje es la cantidad máxima de tokens (palabras o subpalabras) que el modelo puede considerar a la vez como entrada. Define el alcance del "pasado" que el modelo puede usar para predecir la siguiente palabra o generar texto.

Si la secuencia es más **CORTA**
que la ventana de contexto



Padding (relleno)

Si la secuencia es más **LARGA**
que la ventana de contexto



Truncado

Truncado de secuencias:



Si tenemos una secuencia de tamaño M y una ventana de contexto de tamaño N tal que $M > N$, entonces podemos generar $1 + M - N$ secuencias de tamaño N :

Ejemplo ($M=6$, $N=3$)

["Todas", "las", "hojas", "son", "del", "viento"] se convierte en:

["Todas", "las", "hojas"]
["las", "hojas", "son"]
["hojas", "son", "del"]
["son", "del", "viento"]

Data Augmentation y padding



Queremos que el modelo sea capaz de predecir cada elemento de la secuencia, no solo el último:

["Todas", "las", "hojas", "son", "del", "viento"] → [1, 2, 3, 4, 5, 6]

Ejemplo (M= 6, N=3)

["Todas", "las", "hojas"]

["las", "hojas", "son"]

["hojas", "son", "del"]

["son", "del", "viento"]

["Todas"] → [0, 0, 1]
["Todas", "las"] → [0, 1, 2]
["Todas", "las", "hojas"] → [1, 2, 3]